



Veerorx AI

Production Execution Plan

Multi-Channel Voice + WhatsApp AI Agent

Prepared 29 May 2026 · Confidential

1. Executive Summary

Veerox AI is a multi-channel conversational agent. Customers reach the business by **phone call** (voice) or **WhatsApp message**, and an AI agent powered by OpenAI GPT-4o answers naturally in **English or Hindi**. It captures leads, books appointments, looks up existing customers, and escalates to a human when needed. Operators monitor and control everything from a Next.js admin dashboard — live transcripts, lead inbox, cost tracking, and a kill-switch to pause the agent instantly.

Architecture principle: **one brain, two mouths** — a single channel-agnostic AgentCore performs all reasoning; voice and WhatsApp are thin transport adapters around it. This prevents the two channels from drifting apart over time.

Where the project stands today

Component	Status	Note
Agent brain (GPT-4o + tools + memory)	DONE	Fully implemented & tested
WhatsApp channel (in/out + voice notes)	DONE	Awaiting Meta credentials only
Admin dashboard (7 pages)	DONE	Polished UI, runs on :3001
Database + Redis + migrations	DONE	Neon Postgres + Redis
Admin API (stats, kill-switch, escalations)	DONE	Complete
Voice channel (Twilio + Realtime)	NOT BUILT	3 stub files — main gap
Outbound dial backend	MISSING	calls.py to build
Cost tracking (pricing constants)	PARTIAL	costs.py missing
Production deployment + CI/CD	NOT DONE	Local only today

Bottom line: the WhatsApp path is production-grade code waiting on credentials. The **voice channel is the main unbuilt feature**. Everything else is hardening and deployment.

2. Technology Stack

Layer	Technology	Why
Backend API	FastAPI (Python 3.12, async)	Async-native; ideal for webhooks + WebSockets
AI — chat	OpenAI GPT-4o	Strong Hindi/English + tool calling
AI — voice	OpenAI Realtime API	Native speech-to-speech, low latency
Transcription	OpenAI Whisper	WhatsApp voice notes → text
Telephony	Twilio Voice + Media Streams	Phone numbers + audio WebSocket
WhatsApp	Meta Cloud API (direct)	Lower cost, full control
Database	PostgreSQL 16 + SQLAlchemy + Alembic	Relational, multi-tenant ready
Cache / state	Redis	Kill-switch, rate limits, idempotency
Frontend	Next.js 14 + Tailwind	Admin control plane
Observability	structlog (JSON) + Sentry	Cloud-agnostic logs + errors
Deploy	Docker + Caddy (auto-TLS) on EC2	Simple single-box production

3. Execution Roadmap

Phased so each phase ends with something demonstrable. Build order: **A** → **(B in parallel)** → **C** → **D**.

Phase A — Finish the feature set (~1 week)

#	Task	File(s)
A 1	Build voice TwiML webhook	channels/voice/twilio_webhook.py
A 2	Build Realtime audio bridge (WebSocket + interruption via mark/clear)	channels/voice/realtime_bridge.py
A 3	Voice tool-call bridging (reuse same DISPATCH_TABLE)	channels/voice/adapter.py
A 4	Stream voice transcripts to DB during the call	realtime_bridge.py
A 5	Outbound dial endpoint	routers/calls.py + main.py
A 6	Pricing constants + USD-spend calc	core/costs.py

Acceptance: call the Twilio number → English/Hindi conversation → capture_lead fires → interrupting mid-sentence stops playback in ~200ms → transcript rows appear *during* the call.

Phase B — Credentials & live integration (parallel, lead-time heavy)

#	Task
B 1	OpenAI org: confirm gpt-4o AND gpt-4o-realtime are enabled
B 2	Twilio: buy/port India number (KYC 1–3 days), set Voice webhook
B 3	Meta: verify business, create WA app, get permanent token + Phone Number ID
B 4	Submit 2 WhatsApp utility templates (24–48h approval; expect 1 rejection)
B 5	Register webhooks (WA + Twilio) against a stable HTTPS domain
B 6	Fill empty Twilio + Meta keys in .env

Phase C — Production hardening (~1 week)

#	Task	Why
C1	Secrets → AWS SSM Parameter Store	No secrets on the box
C2	Caddy reverse proxy + auto-TLS	Meta/Twilio require HTTPS
C3	Enable webhook signature validation (Meta + Twilio)	Block spoofed requests

#	Task	Why
C4	Rate limiting per channel + per user phone	Abuse / cost guard
C5	Graceful LLM-failure fallbacks (chat + Realtime)	Never crash on 5xx
C6	Idempotency on WhatsApp message IDs	Meta retries → no double reply
C7	Sentry DSN populated + tested	Error visibility
C8	uvicorn --workers 2 behind Caddy	Concurrency (voice WS pins a worker)
C9	Nightly pg_dump → S3 backup	Disaster recovery
C10	Per-call cost counter (audio seconds) to DB	Catch stuck sessions in \$

Phase D — Ops, CI/CD & observability (~3–4 days)

#	Task
D 1	GitHub Actions: ruff + mypy + pytest on every PR
D 2	Docker build + push to registry on merge to main
D 3	Deploy step (SSH/compose pull) or watchtower auto-update
D 4	Health checks (/health + /ready) wired to uptime monitor
D 5	Cost + latency dashboard fed with real data
D 6	Ship stdout JSON logs to CloudWatch/Loki
D 7	Runbook: rotate keys, pause agent, restore backup

Phase E — Scale & Phase-2 (later, out of MVP scope)

Multi-tenant org switching · concurrency/load testing · call-recording playback · knowledge-base RAG (pgvector) · durable jobs (arq/Celery) · real auth (OAuth/SSO) replacing the static admin token · browser-based voice (LiveKit).

4. Production-Readiness Checklist

Security

- Secrets in SSM, never in repo or plaintext on disk
- Both webhook signatures (Meta + Twilio) validated
- Admin token replaced with real auth, or strong rotated token + IP allowlist
- Rate limits live on all public endpoints
- OpenAI keys rotated (the two exposed during dev MUST be rotated)

Reliability

- Graceful fallback on every external API failure
- Idempotency on inbound webhooks
- DB migrations run automatically on deploy
- Nightly backups verified by a test restore

Observability

- Structured logs shipping to an aggregator
- Sentry capturing errors
- Cost + latency visible on dashboard
- Uptime monitor on /ready

Delivery

- CI green (lint + types + tests) required to merge
- One-command deploy
- Rollback path documented

5. Risks & Mitigations

Risk	Impact	Mitigation
OpenAI Realtime cost (~\$1.50 / 5-min call)	Budget blow	Per-call \$ counter + kill-switch + alerts
Meta template rejection	Launch delay	Submit Day 1; expect 1 rejection cycle
Twilio↔OpenAI audio format mismatch	Garbled audio	Use g711_ulaw both sides (no resampling)
Webhook >15s → Meta retries	Duplicate replies	Fast-ACK + background task (done for WA)
Ephemeral voice transcripts	Data loss	Stream to DB during call (Phase A4)
Single EC2 box	Single point of failure	Phase-2 ALB + 2 instances; monitor + backup for MVP

Build order: **A** → **(B in parallel)** → **C** → **D**. This document is the source of truth for taking Veerox AI from “WhatsApp working locally” to “voice + WhatsApp running in production.”